

Programació Dinàmica: El Joc del 31

Josep Oliver Vallespir, Hugo Valls Sabater

Resum—En aquesta pràctica hem dissenyat i implementat una solució amb programació dinàmica i memoització per al Joc del 31. Al document expliquem com hem plantejat l'algorisme per fer-lo compatible amb un nombre N de jugadors, com hem passat de tenir un munt de matrius de tipus primitius lletges a utilitzar objectes ben fets gràcies a la Programació Orientada a Objectes (EstatCalculat), i tota l'arquitectura amb el patró Model-Vista-Controlador (MVC). També comentem detalls que fan que el joc vagi molt millor, com el fet de mesclar el teclat de forma aleatòria, la detecció de quan algú ja ha perdut abans d'hora i com hem gestionat els fils de fons perquè la interfície no es quedi congelada mentre es fan els càlculs pesats.

Index Terms—Programació Dinàmica, Jocs Combinatoris, Memoització, Inducció cap enrere, Disseny Orientat a Objectes, MVC, Concurrencia, Zugzwang, Teclat dinàmic.



1 Introducció

Quan ens vàrem posar a analitzar el Joc del 31 proposat, vam veure que es un problema típic de la Intel·ligència Artificial. Bàsicament, és un joc on dos o més jugadors van sumant nombres a una calculadora central, començant des d'una suma inicial. Qui es passa del límit (que per defecte està posat a 31), perd la partida automàticament.

El que fa que aquest problema deixi de ser una simple suma i es compliqui de veritat són les restriccions del moviment sobre el teclat. No podem triar qualsevol nombre de l'1 al 9. Quan el jugador anterior prem una tecla, ens restringeix: en el nostre torn només podem prémer botons que estiguin en la mateixa fila o en la mateixa columna que el nombre que s'acaba de prémer, i a més a més tenim prohibit repetir exactament el mateix nombre. Això fa que el taulell gràfic influeixi completament en l'estratègia lògica.

L'objectiu principal de la pràctica és fer un programa que, donat un estat de la partida (la suma actual, quin ha estat el darrer nombre clicat, el límit i quants jugadors som), ens digui qui perdrà si tothom juga perfectament. És a dir, assumint racionalitat perfecta. Aquí és on entra la **Programació Dinàmica (PD)**[1]. Com que des d'una mateixa suma i un mateix darrer nombre es repeteixen els mateixos subproblemes una vegada i una altra, calcular-ho amb una recursivitat simple seria erroni. Guardant els resultats en una taula de memoització aconseguim que el càlcul passi de trigar hores a fer-se pràcticament al instant.

- *Estudiants del Grau en Enginyeria Informàtica a la Universitat de les Illes Balears (UIB).*
Assinatura: Algorismes Avançats.

A part de investigar com resoldre la part matemàtica de l'algorisme, un dels punts on hem hagut d'invertir més hores és en fer un codi net. Normalment, quan es fa memoització, la gent tira pel camí fàcil i crea tres o quatre matrius separades per guardar si un estat està calculat, quins camins té o qui guanya. Nosaltres hem volgut fer-ho bé utilitzant Programació Orientada a Objectes (POO), unint tot això en objectes immutables de la classe EstatCalculat. A part, hem programat una interfície gràfica súper interactiva que respecta de forma estricta el patró Model-Vista-Controlador (MVC) i que llança l'algorisme en un fil a part, aconseguint que l'aplicació vagi finíssima i mai es quedi penjada.

2 Arquitectura del Programa i Patró MVC

2.1 Separació de Responsabilitats i Desacoblament pur

Perquè el projecte estigués ben estructurat i no fos un "codi espagueti", hem separat el codi seguint el patró **Model-Vista-Controlador (MVC)**. Això ens garanteix que la interfície gràfica no tingui ni idea de com es calculen les coses, i que el motor lògic es pugui provar de forma aïllada sense haver d'obrir cap finestra.

- **Model (model):** Aquí hi ha tota la "xixa" matemàtica i les regles del joc. Està format per Joc, Teclat i la classe interna EstatCalculat. Una norma sagrada que hem seguit és que cap d'aquestes classes importa res de `javax.swing` ni de components gràfics. El model només sap fer càlculs de programació dinàmica i gestionar les coordenades del teclat.
- **Vista (vista):** Representada per la classe Vista, que hereta de JFrame. Només s'encarrega de pintar els botons, els spinners, els rètols i de

rebre les interaccions. Hem aconseguit que el desacoblament de la vista sigui del 100%. Com ho hem fet? Doncs fent que quan el controlador li diu a la vista que es redibuïxi, només li passa tipus de dades primitius. Per exemple, la capçalera del mètode és:

```
public void actualitzarTeclat(int[][] grid, int w, int h, int maxNombre)
```

Fixeu-vos que la vista rep un `int[][]` buid. No rep cap objecte `Teclat`. Això fa que la vista estigui aïllada i s'oblidi del domini del model, complint el que se'ns demana en l'assignatura.

- **Controlador (controlador):** És l'intermediari que ho junta tot. El controlador escolta quan l'usuari prem un botó de la calculadora, actualitza els valors, li demana al model que recalculi la solució i després li injecta els nous textos i colors a la vista de manera asíncrona.

A continuació podeu observar com està organitzada la nostra estructura de paquets del programa:

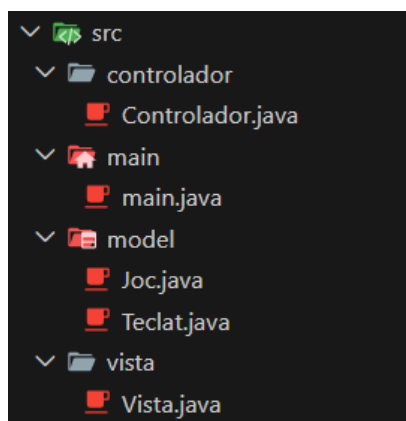


Figura 1. Estructura de Packages del Programa.

2.2 El Cicle de Comunicació Gràfica

La comunicació entre les capes funciona mitjançant esdeveniments per evitar que les classes estiguin agafades mútuament en bucles infinits de referències. El controlador implementa els `ActionListener` i els afegeix als components de la vista mitjançant mètodes de registre, com per exemple `vista.setControladorDimensions`. Quan canvis el desplegable de la mida del taulell, la vista avisa el controlador, el controlador destrueix i torna a crear l'objecte `Teclat` dins del model, agafa la nova matriu d'enters i la hi envia a la vista perquè regeneri la graella de botons a la pantalla.

2.3 Concurrencia i Multi-threading

Un problema greu que tenim quan executam aquests tipus de problemes és la congelació de la pantalla. Al

principi de les proves quan demanàvem a l'algorisme de programació dinàmica que calculés taulells grans (4×4 o 5×5) amb límits alts, la interfície de l'aplicació es congelava durant uns segons. Això passa perquè Java executa tota la part visual en un únic fil anomenat *Event Dispatch Thread* (EDT). Si fem l'algorisme recursiu a córrer dins d'aquest fil, l'EDT es queda ocupat calculant i deixa de pintar la pantalla, donant una sensació súper dolenta.

La solució ha estat programar multi-threading. Cada vegada que s'ha d'executar el mètode `calcularGuanyador()`, el controlador crea un fil nou del sistema amb:

```
new Thread(() -> { ... }).start();
```

Aquest fil de fons s'encarrega d'executar tota la recursivitat i fer córrer la programació dinàmica sense molestar el fil visual. Perquè l'usuari no faci coses rares mentrestant, cridem a `vista.setProcessant(true)`, que bloqueja temporalment els selectors i canvia el punter del ratolí pel típic cercle de càrrega. Quan l'algorisme acaba, utilitzem el mètode:

```
SwingUtilities.invokeLater(() -> { ... });
```

Això torna a passar el control a l'EDT de manera totalment segura per mostrar el final en els rètols de la pantalla i tornar a activar els botons. Amb això aconseguim que el programa sigui súper fluid.

3 Disseny de la Interfície d'Usuari (Vista)

Per fer la interfície gràfica ens hem basat en la llibreria *Swing*. Hem intentat que quedi bastant moderna i neta, allunyant-nos dels dissenys vells de Java [2].

Abans d'explicar cada part aquí tenim una imatge de com es veu la interfície al executar el programa:

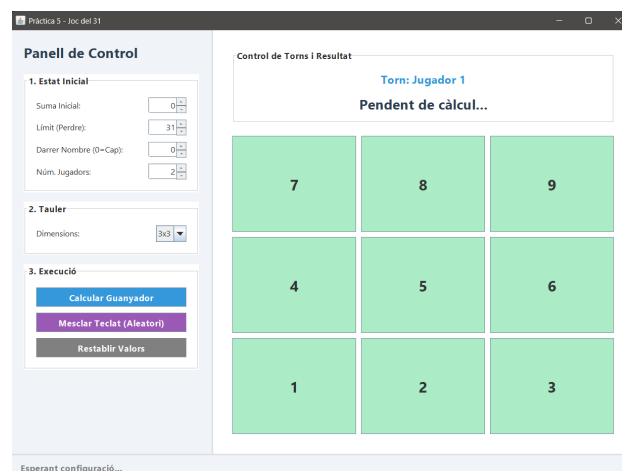


Figura 2. GUI Principal del Programa.

Hem dividit l'aplicació clarament en tres blocs:

3.1 Panell Lateral de Configuració

És la barra esquerra (de color blau-gris molt suau). Aquí hi hem posat unes "targetes" blanques amb vores fines on s'agrupen els selectors:

- **JSpinners per a la configuració numèrica:** Permeten ajustar la *Suma Inicial*, el *Límit* de la partida, el *Darrer Nombre* premut per simular estats intermedis, i el *Núm. Jugadors* (podem canviar entre 2, 3 o 4 jugadors fàcilment).
- **JComboBox per a les mides del taulell:** Deixa escollir configuracions com 2×2 , 3×3 , 4×4 o 5×5 .
- **Botons d'execució:** El botó blau llança l'algorisme de programació dinàmica en qualsevol moment, el gris torna els valors al seu estat inicial, i el botó lila activa la funcionalitat de "Mesclar Teclat", que és una de les variacions que hem dissenyat per anar més enllà del que demanava l'enunciat bàsic.

3.2 Panell Central (El Teclat Interactiu)

Ocupa la major part de la finestra. Aquí es munta el teclat numèric utilitzant un `GridLayout` que s'adapta automàticament a les dimensions triades. El teclat té un sistema visual intel·ligent de colors que canvia dinàmicament a cada torn:

- **Verd Clar:** Pinta les tecles dels moviments vàlids que el jugador actiu pot prémer (estant a la mateixa fila o columna). Estan totalment habilitades per fer-hi clic.
- **Gris Clar/Fosc:** Pinta i deshabilita les tecles que són moviments il·legals segons la posició del darrer nombre, impeding completament que el jugador s'equivoqui de tecla.
- **Blau Clar:** Ressalta quina va ser l'última tecla premuda per l'altre jugador, de manera que ràpidament veus on s'ha quedat el focus de la partida.

A més, a la part superior d'aquest panell, hi apareix informació per saber el torn del jugador actual i qui guanyaria la partida en cas de calcular el jugador guanyador.

3.3 Panell Inferior d'Estat

És una línia de text a la part més baixa de la finestra que actua com a consola d'estat. Ens diu si l'algorisme s'ha calculat bé, i gràcies al model, ens mostra una dada súper curiosa: el nombre exacte de combinacions o rutes de partides possibles que existeixen des de l'estat actual fins a exhaurir totes les opcions del joc.

4 Implementació Tècnica del Model

4.1 L'Algorisme de Programació Dinàmica i Recursivitat a Fons

Anem a explicar detalladament com funciona el cor lògic del projecte, que és on realment ens hem hagut d'aplicar tots els coneixements de l'assignatura.

El nostre problema es pot veure com un Graf Dirigit Acíclic (DAG) d'estats. Un estat complet de la partida queda totalment definit per només dues variables:

- 1) *S*: La suma acumulada actualment ($0 \leq S \leq L$).
- 2) *D*: El darrer nombre escollit que ens diu quines tecles es poden pitjar ara ($0 \leq D \leq M$), on el 0 és un cas especial que significa "començament de la partida" (aquí es pot triar qualsevol botó del teclat sense cap restricció).

Definim la funció recursiva bàsica com $P(S, D)$, que ens retorna el **perdedor relatiu** d'aquell estat. Què vol dir perdedor relatiu? Significa que ens retorna la posició del jugador que acabarà perdent, comptant a partir de qui té el torn en aquest precís moment. És a dir, si retorna 0, vol dir que perdrà el jugador que ha de moure ara mateix. Si retorna 1, perdrà el següent jugador de la ronda, i així successivament. Això és crucial perquè ens permet oblidar-nos d'en quin torn absolut ens trobem i fa que la funció depengui exclusivament de l'estat pur del joc.

L'algorisme utilitza el principi de la **inducció cap enrere**. Anem a desglossar els passos del codi:

- **Els Casos Base (La condició de derrota):** Quan entrem al mètode recursiu, el primer que fem és mirar si ens hem passat del límit. Si la suma actual *S* ja és major o igual que el límit *L* ($S \geq L$), significa que el jugador anterior ens ha deixat la partida ja perduda, o sigui, hem entrat al nostre torn havent perdut ja. Com que el jugador que té el torn no pot fer res i ha perdut, el perdedor relatiu és ell mateix, és a dir, el 0.

$$P(S, D) = 0 \quad \text{si } S \geq L$$

- **El Pas Recursiu i la Triada de Moviments Òptims:** Si la suma és menor que el límit ($S < L$), hem de mirar quines alternatives tenim. Cridem al teclat per saber quins moviments són vàlids a partir de *D*, aconseguint un conjunt de tecles $\{m_1, m_2, \dots, m_k\}$. Per a cada opció vàlida *m*, calculem què passaria en el futur si la triem. Si triem la tecla *m*, la nova suma de la calculadora passarà a ser $S + m$, i el darrer nombre passarà a ser aquesta *m*. Llavors fem una crida recursiva

per avaluar l'estat següent: `avaluarEstat(S + m, m)`.

Aquesta crida ens tornarà un objecte amb el perdedor relatiu de l'estat següent, anem a dir-li $P_{\text{següent}}$. Però alerta, aquest $P_{\text{següent}}$ està calculat des de la perspectiva del jugador que anirà *després* de nosaltres. Com que volem saber què significa això per a nosaltres, hem de fer un desplaçament circular sumant-li 1 i aplicant el mòdul del nombre de jugadors (N):

$$P_{\text{candidat}}(m) = (P_{\text{següent}} + 1) \bmod N$$

Aquesta fórmula és un dels punts clau del nostre disseny. Ens diu exactament qui perdrà la partida en la taula si nosaltres decidim prémer la tecla m .

Ara bé, com triam quin moviment fer d'entre tots els candidats possibles? Com que som jugadors intel·ligents, la nostra prioritat absoluta és salvar-nos nosaltres mateixos. És a dir, volem evitar a tota costa que $P_{\text{candidat}}(m) == 0$, perquè això significaria que perdem nosaltres.

Així doncs, la lògica de decisió que hem programat segueix aquests dos passos:

1) **Intentar salvar-se (Trobar una salvació):**

Cercam si hi ha algun moviment m on $P_{\text{candidat}}(m) \neq 0$. Si en trobem un o més, vol dir que ens podem salvar i fer que perdi un altre jugador. D'entre tots els moviments que ens salven, intentem triar el que ens sigui més favorable o que faci perdre el següent jugador de la seqüència ($P_{\text{candidat}} == 1$) per fotre'l el màxim possible, minimitzant el valor del perdedor relatiu triat.

2) **Derrota inevitable (Zugzwang):** Si resulta que mirem tots els moviments vàlids i en absolutament tots es compleix que $P_{\text{candidat}}(m) == 0$, significa que esteim tancats. No importa quina tecla cliquem, totes les opcions porten a camins on acabarem perdent nosaltres si els nostres rivals juguen bé. Hem entrat en un estat de *Zugzwang* [3]. Com que qualsevol decisió ens duu a la derrota, establim que el millor perdedor relatiu és 0 (nosaltres).

Simultàniament, per saber quantes partides diferents es poden jugar, aprofitem la mateixa estructura de la programació dinàmica per sumar els camins de manera acumulada. Si arribem a un cas base, el camí compta com 1. Si estem en un pas recursiu, el nombre total de partides des de l'estat actual és la suma de totes

les partides que es generen a partir de cadascun dels nostres moviments vàlids disponibles:

$$C(S, D) = \sum_{m \in V(D)} C(S + m, m)$$

4.2 Disseny POO i Memoització: La classe EstatCalculat

Un dels requisits importants d'aquesta pràctica era desmarcar-nos de les clàssiques estructures disperses de tipus primitius. Al principi, la temptació era fer el típic:

```
int[][] dpPerdedor = new int[limit][maxNombre];
long[][] dpPartides = new long[limit][maxNombre];
boolean[][] calculat = new boolean[limit][maxNombre];
```

Però això s'acaba convertint en un calvari de mantenir i és molt poc elegant. Vàrem dissenyar la classe immutable `EstatCalculat`:

```
private static class EstatCalculat {
    private final int perdedor;
    private final long partides;

    public EstatCalculat(int perdedor, long partides) {
        this.perdedor = perdedor;
        this.partides = partides;
    }

    public int getPerdedor() { return perdedor; }
    public long getPartides() { return partides; }
}
```

Gràcies a això, la nostra taula de memoització dins de la classe `Joc` passa a ser una única matriu bidimensional de referències d'objectes:

```
private final EstatCalculat[][] dp;
```

Això ens aporta uns avantatges brutals:

- **Sense matrius booleanes auxiliars:** Com que la matriu guarda referències d'objectes, inicialment totes les cèl·les estan a `null`. Si volem comprovar si un estat ja l'hem calculat abans, només hem de fer: `if (dp[suma][darrerNombre] != null)`. Si és diferent de `null`, ja podem retornar directament el valor guardat en lloc de repetir la recursivitat. Ens estalviem una matriu sencera de booleans.
- **Encapsulació i Cohesió:** Totes les dades que descriuen el comportament final d'un estat (quin jugador és el que acaba caient i quantes rutes terminal s'han obert des d'allà) viatgen juntes dins del mateix objecte en memòria. Si en el futur volguéssim guardar més dades (com quin és el moviment ideal detallat), només hauríem d'afegir un atribut a la classe `EstatCalculat`, sense haver de trencar la lògica de la matriu de la programació dinàmica.

5 Gestió Dinàmica de la Graella Física (Teclat)

5.1 La Disposició Estàndard de Calculadora

Per crear el teclat, hem de tenir en compte que les calculadores tradicionals no tenen l'1 a dalt de tot, sinó a sota (començant des de baix a l'esquerra). A la classe `Teclat.java` hem hagut de fer una traducció geomètrica per transformar els índexs de files (r) i columnes (c) de la matriu física als valors numèrics reals del joc. La fórmula que ens dona el valor exacte és:

$$V(r, c) = (H - 1 - r) \times W + c + 1$$

On W és l'amplada i H és l'alçada triada pel taulell. Si muntem un taulell clàssic de 3×3 , aquesta fórmula ens col·loca automàticament els nombres així:

7	8	9
4	5	6
1	2	3

Figura 3. Teclat del Programa.

5.2 Determinació Espacial de Moviments vàlids en $O(1)$

La norma principal del joc ens diu que només ens podem moure per la mateixa fila o columna del darrer nombre. Una opció hagués estat que cada vegada que algú prem una tecla, ens poséssim a recórrer tota la matriu amb bucles combinats per cercar on estan els veïns vàlids. Però això seria molt ineficient si tenim taulells grans i estem fent milers de crides recursives per segon.

Per solucionar-ho i aconseguir un cost de cerca constant de tipus $O(1)$, hem creat dos vectors directes de memòria anomenats `files` i `columnes`. En el mateix moment en què es construeix el teclat, guardem en aquests vectors la posició exacta de cada nombre. Per exemple, si volem saber la fila física del nombre 6, només hem de consultar `files[6]`.

Gràcies a aquesta indexació immediata, el mètode `getMovimentsValids(int darrerNombre)` pot comprovar de forma instantània quins nombres comparteixen eix visual amb el darrer clicat simplement comparant si

estan en la mateixa fila o columna geomètrica, descartant el propi nombre inicial. Això accelera el rendiment global de l'aplicació de bona manera.

5.3 Barrejat Aleatori de la Graella

Una de les millores opcionals que hem afegit i que ens sembla molt divertida és el **"Teclat Barrejat (Aleatori)"**. Si l'usuari activa aquesta opció, el programa crea una llista amb tots els nombres de la calculadora i els desordena completament utilitzant el mètode `Collections.shuffle()` (que per davall fa servir l'algorisme de barrejat de Fisher-Yates). Després, es col·loquen de forma caòtica en la matriu.

El millor de tot és que tota la nostra estructura de moviments vàlids en $O(1)$ segueix funcionant a la perfecció. Com que actualitzem els vectors de coordenades files i columnes amb la nova disposició barrejada, la restricció espacial de "mateixa fila i mateixa columna" es calcula basant-se en els nous veïns gràfics desordenats. Això fa que l'estratègia del joc canviï per complet en qualsevol moment de la partida; un nombre que abans era súper segur perquè estava protegit ara es pot convertir en una trampa mortal, oferint una experiència de joc completament diferent.

A continuació, una imatge del teclat barrejat amb el "3" com a darrer valor premut:

1	8	9
6	3	5
2	4	7

Figura 4. Teclat Barrejat Aleatòriament del Programa.

6 Generalització a N Jugadors i Lògica de Joc

6.1 Gestió de Torns Circulars per a N Jugadors

Gairebé totes les implementacions típiques que es troben per internet d'aquest joc estan pensades exclusivament per a 2 jugadors. Un dels reptes més xulos d'aquesta pràctica ha estat generalitzar l'algorisme per a un nombre N arbitrari de jugadors ($N \geq 2$), permetent partides de 3 o 4 persones alhora.

El flux de torns segueix una estructura circular clàssica: si juga el jugador k , el següent torn va per al jugador $(k \bmod N) + 1$. Com hem vist a la secció del model, la gràcia és que gràcies a l'ús de la fórmula de desplaçament circular basada en el perdedor relatiu de la programació dinàmica, l'algorisme sap mapejar perfectament qui és el jugador real (absolut) de la taula que s'acabarà menjant la derrota, independentment de si som 2, 3 o 4 jugadors jugant al mateix temps.

6.2 Detecció anticipada d'estat de derrota

En el disseny tradicional del joc, els jugadors havien d'anar pitjant botons manualment fins que la suma de la calculadora superés físicament el límit. Això significava que quan un jugador es quedava acantonat sense cap escapatòria, es veia obligat a fer un clic sabent cert que perdria, fent que les últimes fases del joc fossin una mica absurdes i mecàniques.

Per això, hem implementat un sistema de detecció anticipada de *Zugzwang* al `Controlador.java`. Després de cada moviment realitzat a la interfície gràfica, el programa es posa a mirar el futur immediat de les tecles vàlides del jugador que va ara. Si detecta que totes les tecles permeses per la restricció espacial fan que la suma superi o iguali el límit, el programa s'avança als fets: bloqueja el teclat a l'instant, es posa en vermell i anuncia directament: *"Jugador X no té moviments segurs! HA PERDUT"*. Això estalvia moviments i feim que l'algorisme sigui millor.

6.3 Reinici sense Càlculs Residuals

Un problema que tinguérem durant el desenvolupament de la interfície gràfica amb Swing va ser el botó de *"Restablir Valors"*. Quan premíem aquest botó per netejar la partida, havíem de tornar a posar els spinners a zero, el desplegable a 3×3 , etc. Què passava? Com que el mètode `setSelectedItem` del desplegable i els canvis de valors envien esdeveniments asíncrons a la cua de Swing, aquests esdeveniments es mesclaven. El controlador interceptava que havíem canviat les mides a mig fer i llançava el fil asíncron de l'algorisme abans d'hora. El resultat era que la pantalla es quedava amb textos vells, embrutant el resultat del reinici.

Per solucionar aquest problema de concurrència visual, vàrem introduir un flag de control anomenat `ignorarEvents`. Quan algú fa clic a restablir, el primer que fa el controlador és activar `ignorarEvents = true;`. A partir d'aquí, anem netejant els components de la vista un per un. Si Swing llança algun esdeveniment per sota durant la neteja, el controlador el rep, mira el flag,

i surt immediatament sense calcular res. Un cop hem acabat amb tota la neteja, utilitzem el truc de passar un fil al final de la cua de Swing:

```
SwingUtilities.invokeLater(() -> {
    ignorarEvents = false;
});
```

Això ens assegura de forma robusta que el flag es torna a desactivar només quan la interfície ja s'ha netejat i redibuixat per complet, evitant qualsevol tipus d'error residual.

7 Optimització i Correctesa de la Resolució

7.1 Garanties de l'estratègia

Com que en aquest joc de la calculadora no hi ha daus, ni cartes, ni cap tipus d'element de sort, tot depèn de l'habilitat pura dels jugadors. És el que s'anomena un joc combinatori d'informació perfecta. L'algorisme de programació dinàmica que hem fet és totalment correcte i matemàticament exhaustiu perquè l'arbre de decisions s'explora de cap a cap.

Gràcies a la inducció cap enrere, quan diem que un jugador guanya, no és una estimació o una aposta: és una certesa matemàtica absoluta que si aquest jugador segueix els passos que tria la PD, guanyarà tant sí com no, fàci el que fàci el seu oponent.

7.2 Impacte de les restriccions espacials del joc

L'anàlisi dels estats ens demostra la importància de la disposició bidimensional del teclat. En moltes ocasions, un jugador pot anar guanyant pel que fa a la puntuació numèrica, però la restricció espacial de files i columnes el deixa completament bloquejat. L'algorisme de memoització detecta aquestes situacions al moment: quan un estat es guarda amb perdedor relatiu 0 havent-hi una suma baixa, reflecteix que la disposició geomètrica de les tecles ha asfixiat la llibertat de moviment del jugador actiu.

8 Cas Pràctic: Funcionament Pas a Pas

Perquè es vegi ben clar com funciona tota la lògica que hem estat explicant, anem a analitzar un exemple pràctic, d'aquells que ens han servit a nosaltres per fer les proves de depuració a la terminal de l'ordinador.

8.1 Escenari en un Teclat Estàndard 3×3

Imaginem que estem jugant amb el teclat estàndard que hem esmentat anteriorment:

I suposem que la partida es troba en un estat avançat amb aquests paràmetres de configuració:

- **Límit de derrota:** 31
- **Nombre de jugadors:** 2 (Jugador 1 i Jugador 2)
- **Suma central acumulada actual:** 28
- **Darrer nombre premut anteriorment:** 6 (que surt pintat en blau clar a la pantalla)

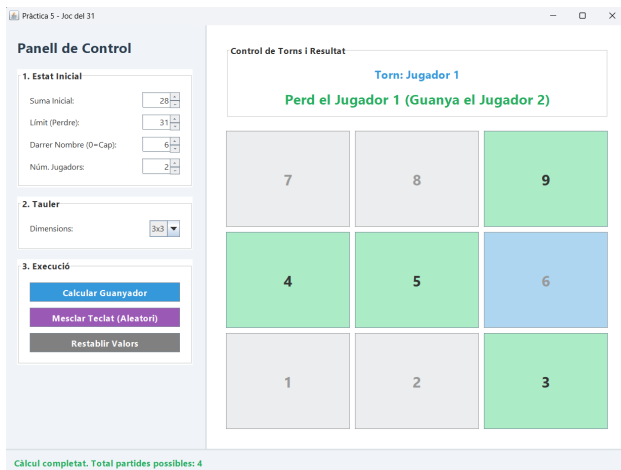


Figura 5. Estat Inicial Calculat.

Ara mateix és el torn del **Jugador 1**. Anem a veure com ho processa el programa per dins.

8.2 Determinació de Moviments vàlids

El darrer nombre premut és el 6. Si mirem la graella de la calculadora, el 6 està situat a la fila del mig i a la columna de la dreta. Per tant, els seus veïns vàlids en la mateixa fila són el 4 i el 5, i en la mateixa columna són el 9 i el 3. El conjunt de moviments permesos que té el Jugador 1 sobre la taula és:

$$V(6) = \{3, 4, 5, 9\}$$

8.3 Avaluació del Torn i Deteció de Zugzwang

El Jugador 1 es posa a analitzar quines conseqüències té prémer cadascuna d'aquestes 4 tecles que té permeses:

- 1) **Si prem la tecla 3:** La calculadora farà la suma $28+3 = 31$. Com que s'arriba exactament al límit de derrota, el Jugador 1 perd immediatament la partida.
- 2) **Si prem la tecla 4:** La suma passa a ser $28+4 = 32$. Ens passem del límit, per tant, perd.

- 3) **Si prem la tecla 5:** La suma passa a ser $28+5 = 33$. Supera el límit, perd.
- 4) **Si prem la tecla 9:** La suma és $28+9 = 37$. Supera el límit, perd.

Com podem veure, **tots** els moviments vàlids sense excepció que té disponibles el Jugador 1 el porten directament a una situació on la nova suma és igual o superior al límit de 31. El Jugador 1 es troba agafat de peus i mans en un estat pur de *Zugzwang* (Posicions pitjors de la que està actualment).

Gràcies al codi que hem posat per fer la detecció primerenca, l'aplicació s'adona d'això abans que el Jugador 1 hagi de triar a la força un botó destructiu. El controlador atura el joc de manera automàtica: canvia el teclat a color gris fosc, posa l'estat en color vermell indicant **"Fi del joc (Derrota inevitable)"**, i pinta en lletres grans a la targeta de resultats: **"Jugador 1 no té moviments segurs! HA PERDUT."**

A continuació, tenim la imatge de final d'aquesta partida, podem veure com no hi ha possibilitat per el Jugador 1 de guanyar la partida:

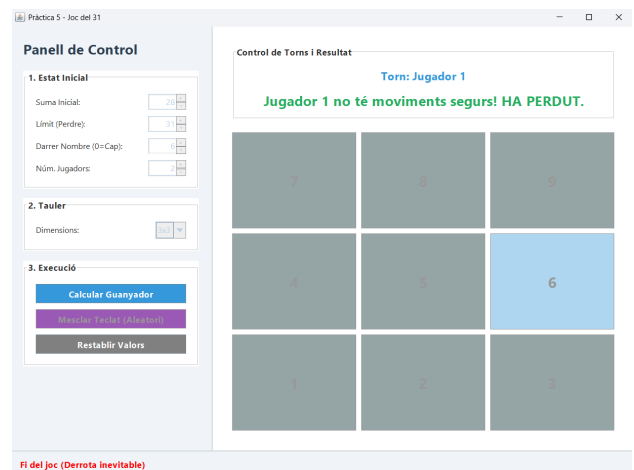


Figura 6. Estat de Final de Partida.

Això demostra de forma pràctica que el motor de l'algorisme i el disseny de la interfície funcionen exactament al 100% de forma sincronitzada.

9 Anàlisi de Complexitat

9.1 Complexitat Temporal

Si haguéssim implementat aquest algorisme utilitzant una recursivitat simple i lletja, sense guardar cap dada, el cost temporal seria una catàstrofe de tipus exponencial, aproximadament de $O(4^L)$, on L és el límit de la partida. Això passaria perquè des de rutes diferents acabaríem calculant exactament les mateixes sumes

milers de vegades, fent que per a límits superiors a 40 l'ordinador es quedés penjat calculant de per vida.

Gràcies a l'ús de la **Programació Dinàmica amb memoització**, aconseguim que cada estat (S, D) es calculi exactament una sola vegada. Anem a fer els nombres de quants estats diferents tenim en el pitjor dels casos:

- Hi ha L valors possibles per a la suma acumulada ($0 \leq S \leq L$).
- Hi ha M valors possibles per al darrer nombre premut ($0 \leq D \leq M$), on M és el nombre total de botons del teclat (9 botons per a 3×3 , 16 botons per a 4×4).

Això ens dona un total màxim de $L \times (M+1)$ estats únics a la taula. Com que per a cada estat que no està calculat hem de fer un petit bucle per recórrer els M moviments vàlids del teclat, la complexitat temporal final del nostre algorisme és polinomial:

$$T(L, M) = O(L \times M^2)$$

Si agafem els valors clàssics de la pràctica ($L = 31$ i un teclat de 3×3 on $M = 9$), el nombre total d'estats de la taula és inferior a $31 \times 10 = 310$ cèl·les. Això es calcula a la CPU de la màquina virtual de Java en menys d'un mil·lisegon. L'execució és literalment instantània i súper eficient.

9.2 Complexitat Espacial

L'espai que ocupa la nostra solució a la memòria RAM de l'ordinador depèn directament de la mida de la graella de memoització que hem creat. Com que hem aconseguit unificar-ho tot en una única matriu bidimensional de referències d'objectes `EstatCalculat[L][M + 1]`, la complexitat espacial del sistema és lineal respecte a les variables d'entrada.

10 Conclusió

Haver fet aquesta pràctica del Joc del 31 ens ha servit un munt per entendre com d'important és dissenyar els programes utilitzant patrons d'arquitectura nets i estructures de dades ben pensades quan ens enfrontem a algorismes complexos. No ens hem volgut quedar només a resoldre la lògica a nivell bàsic, sinó que hem anat un pas més enllà per aconseguir un codi molt més correcte.

En primer lloc, hem aplicat conceptes clàssics de la Programació Orientada a Objectes (POO) dins de les estructures de la Programació Dinàmica. Substituir la típica solució lletja de tenir tres o quatre matrius

de tipus primitius disperses funcionant en paral·lel per una única matriu ben neta d'objectes immutables `EstatCalculat` ha fet que fos una mica més senzill.

En segon lloc, estem súper satisfets d'haver aconseguit un desacoblament real del 100% en el patró de disseny MVC. Fent que la vista només rebí matrius d'enters primitius i variables bàsiques, hem aconseguit que la capa visual sigui totalment independent de la lògica del model. Això ho hem rematat programant una gestió de concurrència amb fils en segon pla, utilitzant flags de control com `ignoreEvents`.

Finalment, detalls com la creació del teclat dinàmic desordenat de forma aleatòria o l'aturada automàtica quan es detecta un estat de **Zugzwang** demostren que els algorismes asimptòtics intensius i la programació dinàmica avançada es poden integrar a la perfecció dins d'entorns interactius del món real. Creiem que hem assolit una aplicació ràpida, divertida i d'una gran qualitat tècnica, complint amb tot el que se'ns demanava afegint-hi els apartats extra.

Referències

- [1] Assignatura d'Algorismes Avançats, Universitat de les Illes Balears, 2026.
- [2] *Gemini*, model 3.5 Pro, Google. Asistència en Swing i LaTeX.
- [3] Zugzwang, Wikipedia La enciclopedia libre. <https://es.wikipedia.org/wiki/Zugzwang>.